NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007: Data Structures and Algorithms

Priority Queues



Presented by Dr Owen Noel Newton Fernando

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Today, we'll examine Priority Queues, a data structure used when the order of processing should depend on importance or urgency rather than simply on arrival time. You're already familiar with normal Queues and their First-In, First-Out behaviour, so we'll use that knowledge as our starting point. We'll explore how priorities change the processing order, examine the main Priority Queue operations, and trace how the structure can be implemented using a Linked List. We'll then connect these ideas to practical applications such as CPU job scheduling, event simulation, and air-traffic control.



Objectives

- Introduce the concept, purpose, and characteristics of a Priority Queue.
- Explain the differences between a Priority Queue and a normal Queue.
- Discuss minimum- and maximum-priority Queues and their ordering principles.
- Discuss the implementation of a Priority Queue using a Linked List.
- Explain the core operations supported by a Priority Queue.
- Introduce common computing and real-world applications of Priority Queues.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This lecture introduces Priority Queues as a data structure for processing elements according to priority rather than arrival order. Students will learn their purpose, main characteristics, and the situations in which they are more suitable than normal Queues.

The lecture also covers minimum- and maximum-priority Queues, including how each type determines which element is processed first. A comparison with the first-in-first-out behaviour of a normal Queue will make these ordering principles easier to distinguish.

Students will then explore the core Priority Queue operations and an implementation based on a Linked List. Computing and real-world applications will connect these ideas to practical uses, such as process scheduling and handling urgent tasks.

Learning Outcome

By the end of this lecture, students should be able to:

- Explain the concept and key characteristics of a priority queue.
- Differentiate between a normal queue and a priority queue.
- Distinguish between minimum-priority and maximum-priority queues.
- Implement a priority queue using a linked list.
- Perform and trace the `peek()`, `enqueue()`, `dequeue()`, `isEmpty()` and `getSize()` operations.
- Identify appropriate computing and real-world applications of priority queues.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

By the end of this session, you should be able to explain how a Priority Queue organises elements and why priority can take precedence over arrival order. You should also be able to compare it with a normal Queue and distinguish between minimum- and maximum-priority behaviour.

Beyond describing these ideas, you'll learn how to implement a Priority Queue using a Linked List. We'll trace each operation carefully, including enqueue, dequeue, peek, isEmpty and getSize, so that you can follow how the structure changes step by step.

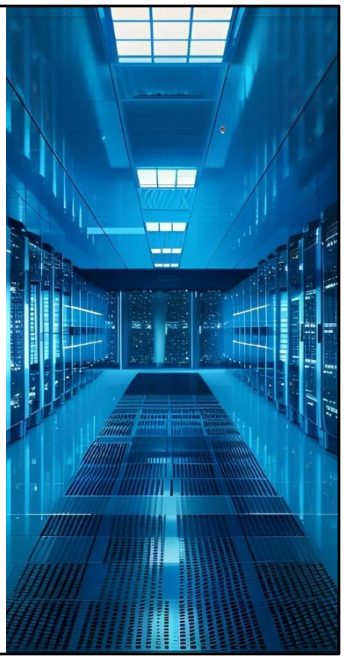
Finally, you should be able to recognise problems where priority-based processing is appropriate. This means connecting the data structure to computing and real-world situations, then deciding whether a Priority Queue fits the required processing order.

Contents



- Introduction to Priority Queues
  - Why Do We Need a Priority Queue?
  - Priority Queue vs. Normal Queue
  - Types of Priority Queues
  - Common Priority Queue Implementations
- Implementing a Priority Queue Using a Linked List
- Core Priority Queue Operations
- Applications of Priority Queues
- Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted



This lecture follows a clear path from the basic idea of a Priority Queue to the way it is used in computing. We'll first consider why priorities are needed, then compare this structure with a normal Queue and examine its main types and characteristics.

The middle part focuses on implementation, where we'll build a Priority Queue using a Linked List. We'll also work through peek, enqueue, dequeue, isEmpty and getSize, paying attention to how each operation changes or inspects the structure.

Towards the end, we'll connect the data structure to practical applications and consider why its ordering behaviour suits particular problems. A short conclusion will then bring the main ideas together and prepare you to apply them.

Introduction to Priority Queues

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We'll begin by exploring what a Priority Queue is and why some tasks need to be processed before others. This will help us understand how priority, rather than arrival order alone, determines which element is handled next.

Introduction to Priority Queues

- A normal Queue processes elements according to the **First-In, First-Out (FIFO)** principle.
- However, in many real-world situations:
  - Processing strictly according to arrival order may not be appropriate.
  - Some items may need to be processed before others.
  - The processing order may depend on the importance or urgency of each item.
  - We therefore need a mechanism for assigning priorities to elements.
- This leads to the Priority Queue.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Picture an ordinary queue at a service counter: the person who arrives first is served first. A normal Queue follows exactly this FIFO rule, which works well when every element should be treated equally.

Computing systems, however, often handle tasks with different levels of urgency or importance. If an urgent task arrives later, following arrival order strictly could delay work that needs immediate attention.

A Priority Queue addresses this limitation by associating each element with a priority that influences when it is processed. Rather than replacing the idea of a Queue completely, it changes the selection rule, allowing the most relevant element to be handled before less urgent ones.

Why Do We Need a Priority Queue?

- Consider an **air-traffic control system**.
- Several aircraft may be waiting to land.
- Should they always land according to the order in which they arrived?
- Not necessarily. The decision may depend on:
  - Distance from the runway
  - Time already spent waiting
  - Remaining fuel
  - Other operational priorities
- Therefore, an aircraft arriving later may sometimes need to be processed **before** one that arrived earlier.
- Order of arrival is not always the best way to determine processing order.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Imagine several aircraft waiting to land while an air-traffic controller decides which one should use the runway next. Serving them strictly in arrival order may appear fair, but it could ignore conditions that make one aircraft more urgent.

An aircraft with little fuel, for instance, may need to land before another that arrived earlier. Controllers may also consider distance from the runway, waiting time, weather conditions and wider operational requirements when assigning priority.

This is exactly the kind of situation a Priority Queue is designed to represent. Each aircraft can be treated as an element with an associated priority, allowing the system to select the most urgent case rather than simply the earliest arrival.

What Is a Priority Queue?

- **Priority Queue** is an Abstract Data Type (ADT) that stores **prioritised elements**.
  - Each element is associated with a **priority**.
  - Elements can be inserted into the Priority Queue.
  - The element with the **highest priority** is selected for processing.
  - Therefore, elements are not necessarily processed according to their arrival order.
- For example:
  - **Priority 1 → processed before Priority 2 → processed before Priority 3**

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A Priority Queue is best understood as a rule for deciding which stored element should be processed next. As an Abstract Data Type, it defines the behaviour we expect without requiring one particular implementation.

Every inserted element carries a priority value as well as its data. When an element is requested for processing, the Queue selects the one considered highest priority, even if it entered after several others.

On this slide, a smaller number represents a higher priority, so Priority 1 is processed before Priority 2 and Priority 3. That numbering convention must be stated clearly because other systems may use larger values for higher priority, while the underlying Priority Queue principle remains unchanged.

Priority Queue vs. Normal Queue

Normal Queue	Priority Queue
Follows **FIFO**	Follows **priority order**
First item arriving is processed first	Highest-priority item is processed first
Arrival time determines the order	A priority/key determines the order
New item normally joins at the rear	Item is managed according to its priority

Normal Queue

A → B → C → D

A processed first

Priority Queue

B(P=1) → D(P=2) → C(P=3) → A(P=4)

B processed first

A Priority Queue therefore allows an item that arrives later to be processed earlier when it has a higher priority.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

So, what really separates a Priority Queue from the normal Queue we already know? With a normal Queue, there's no decision to make: A arrived first, so A is processed first, followed by B, C and D.

A Priority Queue works differently because every item carries a priority value. Here, lower numbers represent higher priorities, which means B with Priority 1 is processed before D, C and A, regardless of when each item arrived.

Think of arrival order as one possible rule, rather than a rule we must always follow. When tasks have different levels of urgency, priority gives us a more suitable processing order, although a normal FIFO Queue remains useful when all items should be treated equally.

Types of Priority Queues

Minimum-Priority Queue
(Ascending key order)

- The **smallest key** has the highest priority.
- The item with the smallest key is processed first.
- Processing order:
  7 → 8 → 15 → 20 → 40 → 50
- **7 is processed first**

Maximum-Priority Queue

(Descending key order)

- The **largest key** has the highest priority.
- The item with the largest key is processed first.
- Processing order:

  50 → 40 → 20 → 15 → 8 → 7
- **50 is processed first**

The selected Priority Queue type determines whether the smallest or largest key is processed first.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The two Priority Queue types differ in the key selected for processing first. A minimum-priority Queue gives precedence to the smallest key, whereas a maximum-priority Queue gives precedence to the largest key.

On the left, the minimum-priority Queue processes 7 first and then continues in ascending key order towards 50. On the right, the maximum-priority Queue starts with 50 and continues in descending key order towards 7.

The terms minimum and maximum refer to the key selected, rather than to an item's importance in everyday language. When designing a Priority Queue, we must define what the key represents and choose the appropriate type, because that choice determines whether the smallest or largest key will be processed first.

Common Priority Queue Implementations

- **A queue is termed as a priority queue if it has the following characteristics:**
  - Each item has some priority associated with it.
  - The item with the highest priority is selected and removed first.
- A Priority Queue can be implemented using:
  1. Unordered array
  2. Ordered array
  3. Linked List
  4. Binary heap
  5. Binary search tree

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted



A Priority Queue describes a particular behaviour rather than one fixed arrangement in memory. Each item has an associated priority, and the item judged to have the highest priority is selected and removed first.

That behaviour can be implemented using several underlying data structures, with different trade-offs. An unordered array allows straightforward insertion but requires a search for the highest-priority item, while an ordered array or ordered linked list performs more work during insertion to maintain priority order.

A binary heap offers a useful balance because both insertion and removal can be completed efficiently, while a binary search tree provides another option when its ordering is maintained. For the remainder of this lecture, we'll use an ordered linked list so the highest-priority item remains at the front and can be accessed directly.

Implementing a Priority Queue Using a Linked List

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've seen the behaviour expected from a Priority Queue, but we still need a structure that can support it. In this section, we'll use a Linked List to keep items arranged by priority, allowing the highest-priority item to be accessed and removed from the front.

Implementing Priority Queue using Linked List

```
class PriorityNode:
    def __init__(self, data, priority):
        self.data = data
        self.priority = priority
        self.next = None
```

```
class PriorityQueue:
    def __init__(self):
        self.head = None
        self.size = 0
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Our implementation begins with two small classes, each responsible for a different part of the structure. `PriorityNode` represents one item, storing its data, its priority value and a reference to the next node.

`PriorityQueue` manages the structure as a whole through `head` and `size`. When a new Queue is created, `head` is `None` because no node exists yet, while `size` starts at zero and records the number of stored items.

The diagram shows how this develops once nodes are added, with `head` referring to the first node in the chain. Because smaller values represent higher priorities here, the node with priority 1 appears before the node with priority 4, ready to be processed first.

Core Priority Queue Operations

- **peek():** Inspect the highest priority item in the queue without removing it.
- **enqueue():** Add an item to the queue based on its priority.
- **dequeue():** Remove and return the item with the highest priority from the queue.
- **isEmpty():** Return True when the Queue contains no items.
- **getSize():** Returns the current number of items in the priority queue.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



These five operations form the interface through which we interact with the Priority Queue. The `peek()` operation allows us to examine the highest-priority item without changing the structure, while `dequeue()` returns that item and removes it.

Most of the ordering work takes place during `enqueue()`, because the new node must be positioned according to its priority. In our ascending-order linked list, smaller priority values stay nearer the head, so the next item for processing is immediately available.

The remaining operations support safe and consistent use of the structure. `isEmpty()` lets us check whether an item exists before calling `peek()` or `dequeue()`, while `getSize()` tracks the number of stored items as `enqueue()` and `dequeue()` operations modify the Queue during program execution.

peek()

- Inspect the item at the front of the queue without removing it

```
def peek(self):
        if self.isEmpty():
            raise IndexError("Queue is empty")
        return self.head.data
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Suppose we want to know which item will be processed next, but we're not ready to remove it. That is exactly what `peek()` provides: a read-only view of the data stored in the node at the head.

The method first calls `isEmpty()` because an empty Queue has no head node to inspect. If the Queue is empty, raising an `IndexError` makes the problem explicit and prevents an attempt to access the `data` field through `None`.

Otherwise, the method returns `self.head.data`, which is the data belonging to the highest-priority item in our ordered linked list. Notice that neither `head`, `size` nor any `next` reference changes, so repeated calls to `peek()` return the same result until an `enqueue()` or `dequeue()` operation changes the Queue.

enqueue(): Add an item to the queue based on its priority

```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```

```
class PriorityNode:
    def __init__(self, data, priority):
        self.data = data
        self.priority = priority
        self.next = None
```

```
class PriorityQueue:
    def __init__(self):
        self.head = None
        self.size = 0
```

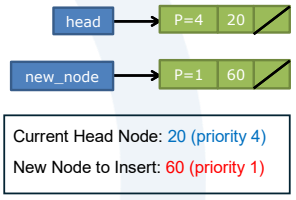


© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The enqueue() operation does more than simply add a new item; it must place that item in the correct position according to its priority. We'll first look at the complete method here, and then trace its execution step by step over the next few slides.

enqueue(): Add an item to the queue based on its priority

```
def enqueue(self, data, priority):
    new_node = PriorityNode(data, priority)
    if self.isEmpty() or priority < self.head.priority:
        new_node.next = self.head
        self.head = new_node
    else:
        current = self.head
        while current.next and current.next.priority <= priority:
            current = current.next
        new_node.next = current.next
        current.next = new_node
    self.size += 1
```

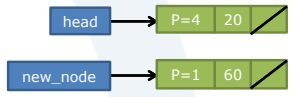


© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Here, the existing Queue contains node 20 with priority 4. We then create new_node, which stores the value 60 with priority 1; at this stage, it has not yet been connected to the Queue.

enqueue(): Add an item to the queue based on its priority

```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```



Current Head Node: 20 (priority 4)

New Node to Insert: 60 (priority 1)

`priority < self.head.priority`

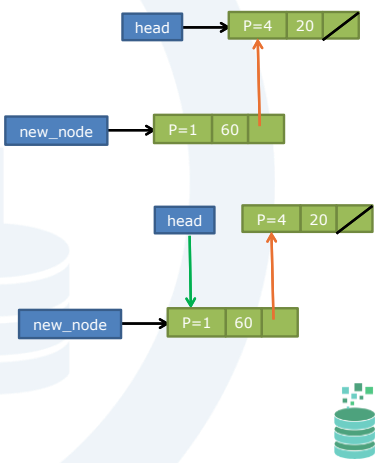
`1 < 4`

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The Queue is not empty, so the second part of the condition compares the new node's priority with the current head's priority. Since **1 is less than 4**, the condition evaluates to True, and execution enters the if block.

enqueue(): Add an item to the queue based on its priority

```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Inside the if block, new_node.next = self.head makes the new node containing 60 point to the existing head node containing 20. This creates the link from 60 to 20 without losing the existing node.

Next, self.head = new_node moves the head reference from node 20 to the new node 60. The Queue is therefore rearranged as 60 with priority 1, followed by 20 with priority 4, so the higher-priority node is now at the front.

enqueue(): Add an item to the queue based on its priority

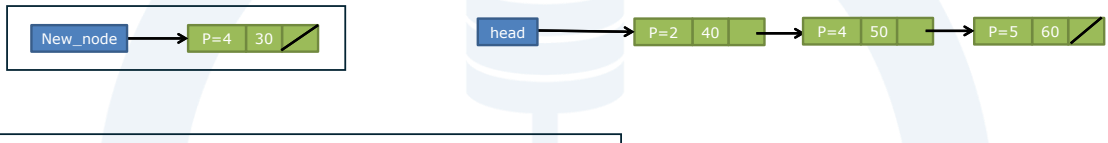
```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Finally, self.size += 1 increases the Queue's size by one because a new node has been added. In this example, the size changes from one to two, matching the two nodes now stored in the Queue.

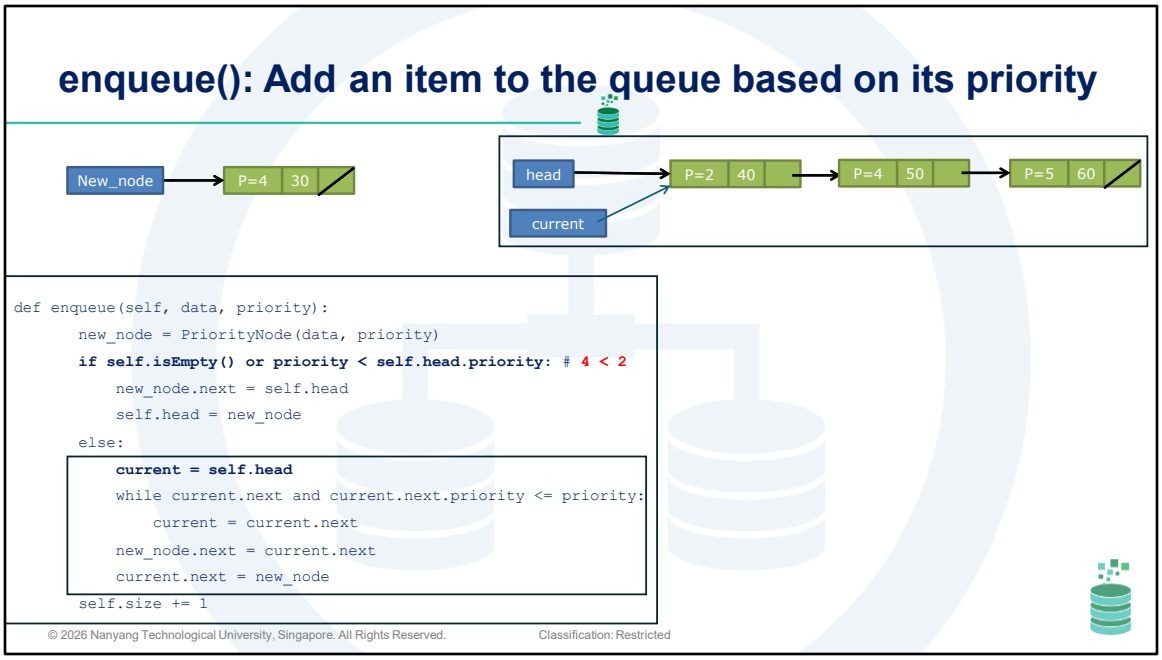
enqueue(): Add an item to the queue based on its priority



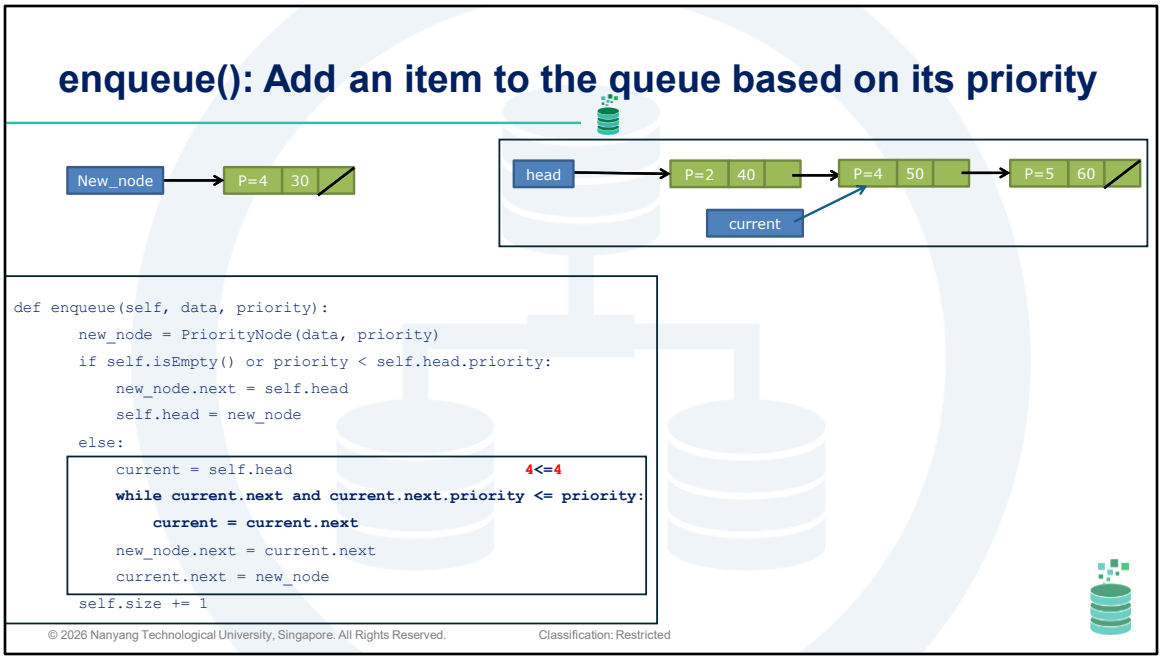

```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here, the existing Queue contains three nodes arranged in ascending priority order: 40 with priority 2, 50 with priority 4, and 60 with priority 5. We create a new node containing 30 with priority 4; at this stage, it remains separate while the algorithm searches for its correct insertion position.

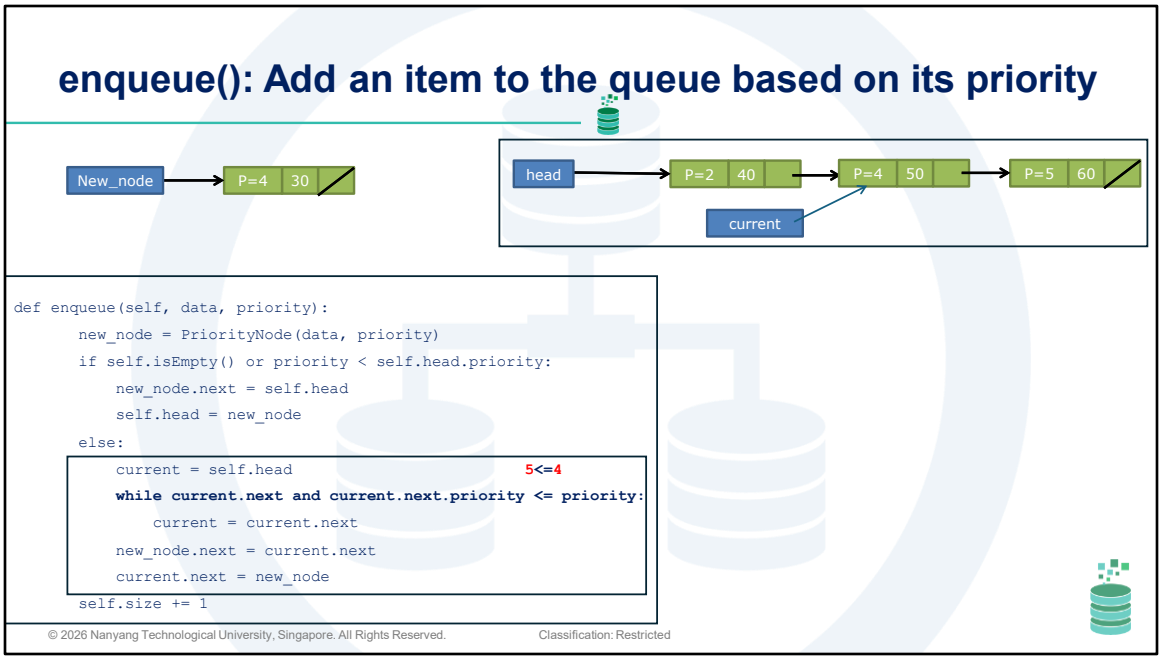


The Queue is not empty, and the new priority 4 is not less than the head priority 2, so the condition evaluates to False. Execution therefore enters the else block, where current = self.head makes current point to the head node containing 40 with priority 2.



The while condition checks that a next node exists and that its priority is less than or equal to the new node's priority. Here, the next node has priority 4, and the new node also has priority 4, so the condition is true.

The statement current = current.next therefore moves current from the node containing 40 to the existing node containing 50. This also ensures FIFO order for equal priorities because the new priority-4 node will be inserted after the existing priority-4 node.

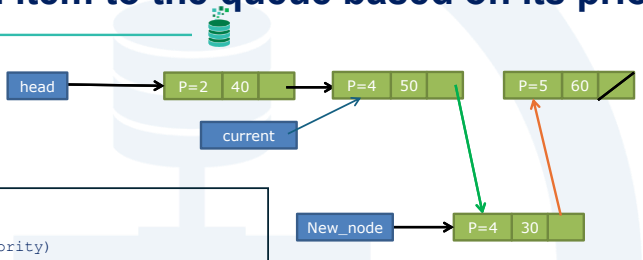


At this point, current refers to the existing node containing 50 with priority 4. The loop checks the following node, which contains 60 with priority 5.

Because 5 is not less than or equal to the new priority 4, the condition evaluates to False. The loop therefore stops, leaving current at the node containing 50.

This tells us that the new node belongs immediately after the existing priority-4 node and before the priority-5 node. The next two statements will update the references needed to insert it at that position.

enqueue(): Add an item to the queue based on its priority



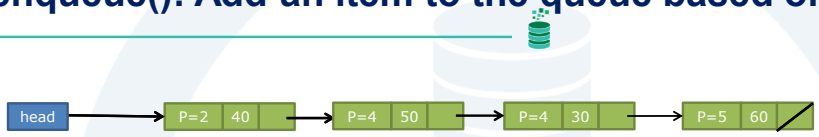

```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The statement new_node.next = current.next first makes the new node containing 30 point to the node containing 60. This preserves the connection to the remaining part of the Queue.

Next, current.next = new_node changes the next reference of the current node containing 50, making it point to the new node containing 30. The complete Queue is therefore connected as 40 with priority 2 → 50 with priority 4 → 30 with priority 4 → 60 with priority 5.

enqueue(): Add an item to the queue based on its priority

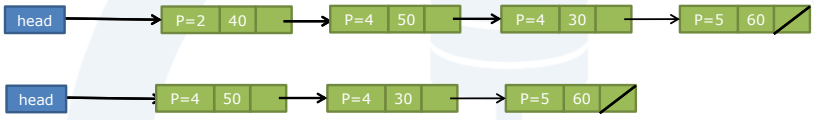



```
def enqueue(self, data, priority):
        new_node = PriorityNode(data, priority)
        if self.isEmpty() or priority < self.head.priority:
            new_node.next = self.head
            self.head = new_node
        else:
            current = self.head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node
        self.size += 1
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Finally, self.size += 1 increases the Queue's size by one to record the newly inserted node. Since the Queue previously contained three nodes and now contains four, its size changes from three to four.

dequeue()




```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Queue is empty")
        removed_data = self.head.data
        self.head = self.head.next
        self.size -= 1
        return removed_data
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Unlike enqueue(), dequeue() doesn't search through the linked list because the highest-priority item is already at the front. The method first calls isEmpty() and raises an IndexError if no item is available.

For a non-empty Queue, removed_data = self.head.data saves the data value stored in the first node. In the diagram, this saves 40, which has priority 2 and is selected for removal.

Next, self.head = self.head.next advances head from node 40 to node 50, making node 50 the new first node. The size decreases from four to three, and the saved value 40 is returned; because the Queue no longer references the removed node, Python can reclaim its memory.

isEmpty()

- Check whether the queue is empty

```
def isEmpty(self):
        return self.head is None
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Although isEmpty() contains only one line, it protects several other Queue operations from working with a node that doesn't exist. The method tests whether head is None because head is our entry point to the entire linked structure.

If head refers to a node, as shown in the diagram, the expression is false and the Queue is not empty. Here, head points to the node containing 40, so the remaining nodes can be reached by following the next references.

When the final node is removed, head becomes None, and the same expression returns true. Methods such as peek() and dequeue() use this result before accessing head.data, while enqueue() uses it to recognise when the first node must be added to an empty Queue.

getSize()

- Returns the current size of the queue

```
def getSize(self):
        return self.size
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Counting the nodes from the head would tell us how many items are present, but repeating that traversal whenever we need the size would be unnecessary. Instead, the Queue maintains a size attribute, and getSize() simply returns its current value.

In the diagram, four nodes are linked together, so self.size should contain the value 4. The size began at zero when the Queue was created, increased after each successful enqueue(), and decreased whenever dequeue() removed an item.

This approach makes the size immediately available without walking through the linked list from beginning to end. However, every operation that adds or removes a node must update self.size correctly; otherwise, the stored count will no longer match the actual number of nodes in the Queue.

Applications of Priority Queues

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Priority Queues are useful whenever some tasks need to be handled before others, regardless of their arrival order. We'll now look at how this behaviour supports both computing systems and familiar real-world processes.

Computing Applications of Priority Queues

- **CPU job scheduling:** Selects the highest-priority waiting job for execution.
- **Event-driven simulation:** Processes the event with the earliest timestamp next.
- **Dijkstra's algorithm:** Selects the unvisited vertex with the smallest tentative distance.
- **Huffman coding:** Repeatedly combines the two nodes with the lowest frequencies.
- **Sorting:** Repeatedly removes elements according to priority to produce a sorted sequence.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Computers often need to select the most suitable item from many waiting candidates as new data and tasks continually enter the system. CPU schedulers can use priorities to choose the next job, while event-driven simulations treat the earliest timestamp as the highest priority so events occur chronologically.

Graph and compression algorithms use the same selection idea with different keys. Dijkstra's algorithm repeatedly selects the unvisited vertex with the smallest tentative distance, whereas Huffman coding repeatedly removes the two nodes with the lowest frequencies and combines them.

Priority Queues can also support sorting by inserting elements and repeatedly removing the minimum or maximum item. Although these applications solve different problems, each depends on quickly identifying the item with the best key at a particular stage of processing.

Real-World Applications of Priority Queues

Priority Queues are useful when items must be processed according to their importance or urgency, rather than their arrival order.

- **Air-traffic control**: Aircraft may be prioritised according to emergency status, remaining fuel, distance from the runway, or waiting time.
- **Hospital emergency departments**: Patients with more serious conditions are treated before those with less urgent conditions.
- **Airline standby lists**: Seat allocation may consider membership status, fare category, and check-in time.
- **Customer-support systems**: Critical requests may be handled before routine enquiries.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Outside computing, we regularly face situations where first-come, first-served is not the safest or most suitable rule. Priority Queues provide a useful model for these situations, although real organisations usually consider several factors when assigning priority.

In air-traffic control, an aircraft with an emergency or limited fuel may need attention before one that arrived earlier. Hospital triage follows a similar principle: clinical urgency guides treatment order, so a seriously ill patient can be seen before someone with a less urgent condition.

Airline standby lists may combine membership status, fare category and check-in time to decide who receives an available seat. Customer-support systems can likewise place service outages or security incidents ahead of routine enquiries, showing how the priority value can represent different criteria while the selection principle remains the same.

Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

To conclude, let’s bring together what we have learned about Priority Queues, their linked-list implementation, core operations and practical applications.

Conclusion



- A Priority Queue stores elements together with their associated priorities.
- Unlike a normal Queue, elements are processed according to priority rather than arrival order.
- A Priority Queue may be a minimum-priority or maximum-priority Queue, depending on whether the smallest or largest key is processed first.
- Its core operations include enqueue(), peek(), dequeue(), isEmpty(), and getSize().
- In our linked-list implementation, elements are maintained in priority order, allowing the highest-priority element to be accessed at the head.
- Priority Queues are widely applied in scheduling, pathfinding, data compression, event simulation, and real-world decision-making systems.

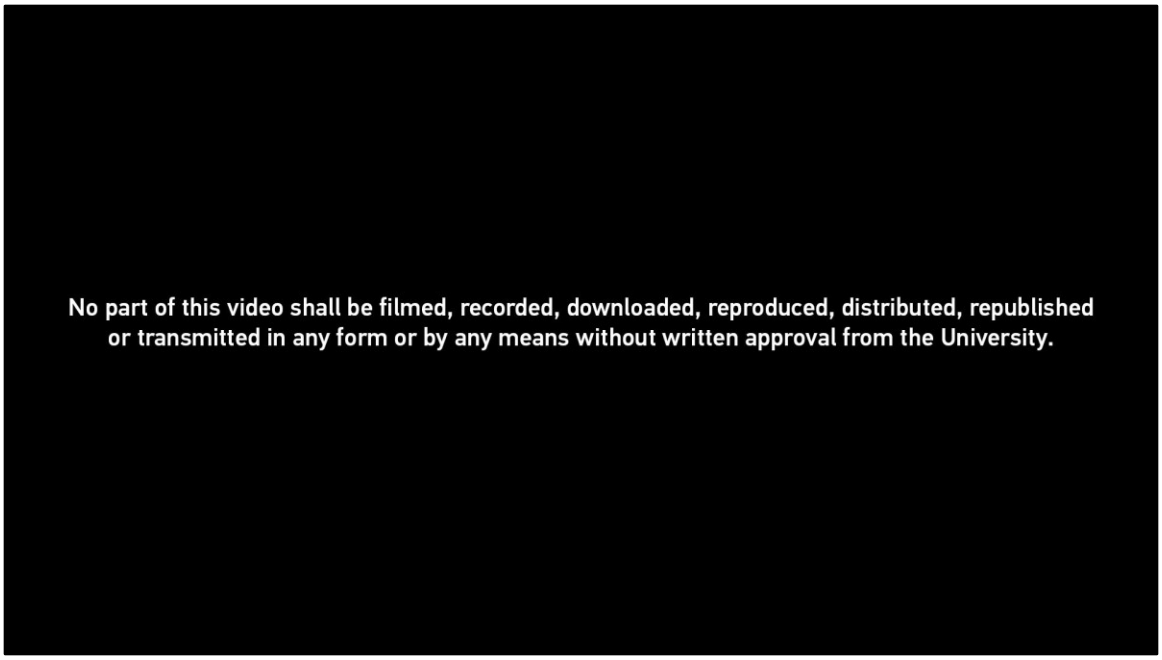


© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At this point, we can see that a Priority Queue is defined by how it selects the next element, not simply by how elements arrive. Pairing each item with a priority allows urgent or relevant work to be processed before items that have waited longer.

The ordering rule may place either the smallest or largest key first, depending on what the priority values mean. In our linked-list version, nodes remain in priority order, so peek() and dequeue() work at the head while enqueue() finds the correct insertion position.

Together with isEmpty() and getSize(), these operations provide a complete interface for managing the structure safely. The same selection behaviour supports scheduling, pathfinding, compression, simulation and real-world decision systems, which is why Priority Queues appear across many areas of computing.


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.